GIRA / ASIC AREA LAB

Del VHDL al área ASIC

Qué medimos y qué podemos demostrar

Código, arquitectura, bibliotecas, comandos y evidencia verificable

11 DPU integradas

3 bibliotecas typical

30 casos documentados

REVISIÓN TÉCNICA · 19 OCTUBRE 2026

La base metodológica es defendible; la validación sigue siendo parcial

Coherencia comprobada

30 reportes: conteos, áreas, hashes, vectores y logs compatibles.

Pendientes reales

LNS16: tres mapeos sin cerrar. No existe certificación física ni prueba formal.

Decisión propuesta

Aceptar el método para área de celdas y cerrar las brechas antes de ampliar las afirmaciones.

No se concluye que todo el sistema esté validado exhaustivamente.

La magnitud de interés es la suma de áreas de las celdas mapeadas

A celdas = Σ [número de instancias × área de su tipo]

SE INCLUYE

- Lógica combinacional
- Flip-flops del wrapper
- ROM reducida a lógica

FUERA DEL ALCANCE

- Área de cables y routing
- Core, pads, die y clock tree
- Potencia, frecuencia y STA

El área física de las conexiones se excluye; las conexiones lógicas se conservan.

El objeto medido son once DPU registradas ya existentes

Perfil	A/B	C/R	Registros declarados	Particularidad
int8_16	8	16	96	Referencia entera
int16_32	16	32	192	Referencia entera
fxp8_16	8	16	96	MAC del RTL
fxp16_32	16	32	192	MAC del RTL
fp8	8	8	80	FMA del RTL
fp16	16	16	160	FMA del RTL
fp32	32	32	320	FMA del RTL
posit8	8	8	80	es=0
posit16	16	16	160	es=1; nombre histórico _0
posit32	32	32	320	es=2
lns16	16	16	160	ROM de corrección

Los bits declarados no se convierten automáticamente en un número fijo de FF mapeados.

El wrapper establece la frontera de área y la latencia observable

80 bits de entrada

A0…A3, B0…B3 y C0

→

DPU combinacional

4 productos + acumulación

→

16 bits de salida

R_out registrado

INT8/16: 8 × 8 + 2 × 16 = 96 bits declarados

Se mide la DPU con su wrapper; no solo la expresión aritmética.

Diseño y tecnología se eligen antes de crear el trabajo

Diseño / formato

Ejemplo: int8_16

→

Tecnología / Liberty

Ejemplo: nangate45

→

POST /experiments

UUID + estado queued

```
POST /api/v1/experiments
{"design_id":"int8_16","technology_id":"nangate45"}
```

Un experimento combina un top y un Liberty; no ejecuta todo el barrido automáticamente.

Las tres bibliotecas activas corresponden a condiciones typical

ID / nodo	Voltaje	Temperatura	Archivo Liberty activo
ihp130 · 130 nm	1,20 V	25 °C	sg13g2_stdcell_typ_1p20V_25C.lib
nangate45 · 45 nm	1,10 V	25 °C	NangateOpenCellLibrary_typical.lib
nangate15 · 15 nm	0,80 V	25 °C	NanGate_15nm_OCL_typical_conditional_nldm.lib

Sí: todos los experimentos documentados usan los typical seleccionados.

Un proceso real y un kit predictivo sostienen conclusiones distintas

IHP SG13G2 · 130 nm

Deriva de un proceso fabricado. El PDK abierto figura como preview.

Nangate / FreePDK45

Biblioteca de referencia sobre un proceso genérico predictivo de investigación.

Nangate / FreePDK15

Referencia predictiva FinFET; no equivale a cualquier nodo comercial de 15 nm.

La conclusión válida nombra la biblioteca exacta, su corner y su procedencia.

El inventario suministrado es mayor que el catálogo habilitado

33 actuales

11 diseños × 3 Liberty activos; todos typical.

121 propuestas

11 diseños × 11 Liberty NLDM base adicionales, sin integrar.

53 archivos inventariados

NLDM, CCS, ECSM y funcionales; se excluyen ._* de Apple.

Tener un archivo en syn_libraries no lo habilita automáticamente en la API.

La aplicación separa decisiones de dominio de herramientas externas



El frontend representa datos; el cálculo está en los adaptadores EDA y Liberty.

Un solo contenedor reúne API, frontend compilado y herramientas EDA

Node + Vite

Construye frontend/dist

→

OSS CAD Suite

Release y SHA fijados

→

Python + FastAPI

Sirve UI, API y worker

Volumen /app/data = experiments.sqlite3 + runs/<UUID>/

Un contenedor; SQLite embebido; un worker EDA. No se necesita PostgreSQL para este alcance.

El trabajo avanza por estados y publica el área al terminar los controles

queued

UUID persistido

→

running

Validación + síntesis

→

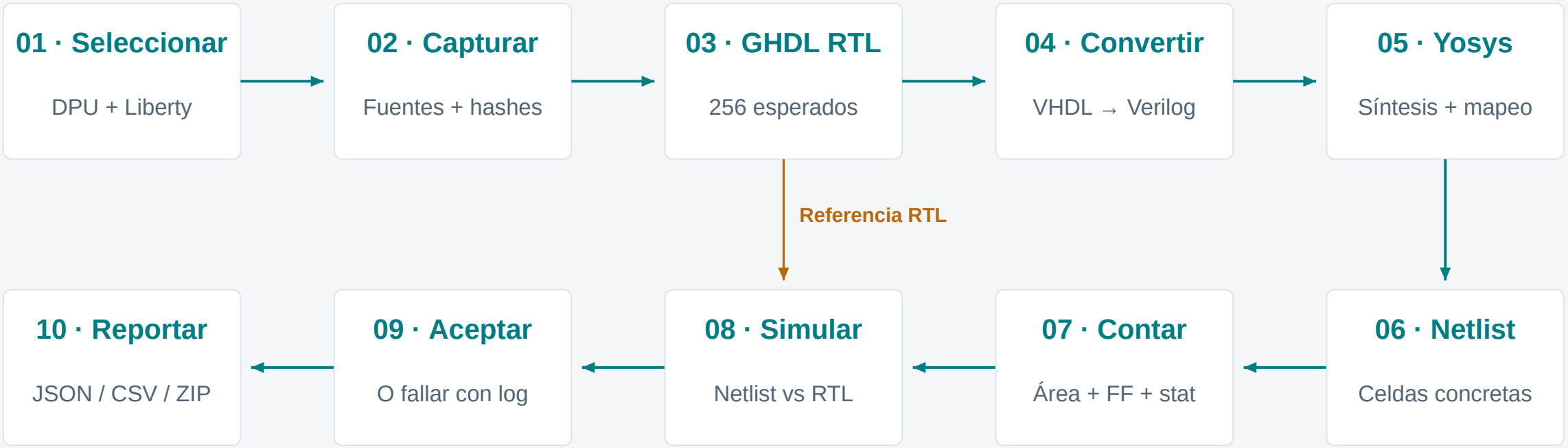
succeeded / failed

Reporte o diagnóstico

```
preparing → analyzing → validating_rtl → converting
→ mapping → checking → validating_mapped → reporting
```

Un exit code correcto de una herramienta no basta para publicar un reporte válido.

Una ejecución conecta entradas, referencia RTL, mapeo y reporte



El reporte se publica tras comprobar estructura, cuenta y comportamiento en los estímulos seleccionados.

El perfil del diseño evita seleccionar archivos o anchos a mano

```
id="int8_16"
top="INT8_16_DPU_perf_top"
library="int8_16_lib"
operand_bits=8; result_bits=16
declared_register_bits=96
reference="signed_dot_modulo"
expected_flip_flops=96
compilation="files"
```

CÓMO INTERPRETARLO

Un único perfil determina:

- El RTL y el wrapper.
- El orden de análisis.
- Los anchos del testbench.
- El control de registros.
- La referencia funcional.

El programa mide variantes revisadas del repositorio; no cualquier VHDL arbitrario.

Cada ejecución conserva la identidad de sus entradas

Originales

RTL + Liberty suministrados

→

assets/

Snapshots y SHA-256

→

runs/<UUID>/

Entradas y manifiesto del caso

Auditoría actual: 44 archivos originales ↔ snapshots, hashes coincidentes

La trazabilidad explica qué se midió; no prueba por sí sola que el RTL sea correcto.

GHDL analiza las dependencias y el wrapper en bibliotecas separadas

```
ghdl -a --std=08 -fsynopsys -fexplicit \
  --work=int8_16_lib <fuentes ordenadas>

ghdl -a --std=08 -fsynopsys -fexplicit \
  sources/rtl/int8_16/INT8_16_DPU_perf_top.vhd
```

CÓMO INTERPRETARLO

Primero: operadores y DPU.

Después: wrapper en work.

Cada caso usa su carpeta y sus bibliotecas compiladas.

Las rutas completas quedan en run.log.

Los comandos se muestran para INT8; las fuentes se sustituyen por las del perfil elegido.

Posit16 y Posit32 requieren resolver helpers duplicados sin alterar su RTL

Suma + multiplicación

Contienen helpers repetidos

→

ghdl --chop

Unidades con nombre y hash

→

Comparar y analizar

Solo reutilizar bytes idénticos

Mismo nombre + contenido distinto → PipelineFailure

Posit16 usa es=1 según sus operadores, aunque sus archivos se llamen posit16_0_*.

El RTL produce la referencia y los enteros tienen un oráculo adicional

Casos reproducibles

256 estímulos por perfil. Semilla 20261005, casos dirigidos y palabras crudas.

INT8 / INT16

Oráculo independiente de producto punto firmado módulo 2^bits(R).

FXP / FP / Posit / LNS

Oráculo del propio RTL. Comparación bit a bit; precisión matemática no evaluada.

La prueba adicional de enteros no se extiende automáticamente a las otras aritméticas.

Dos flancos por estímulo respetan el wrapper registrado

Asignar A/B/C

Entradas estables

→

Flanco 1

Registrar entradas

→

Flanco 2 + muestreo

Registrar y comprobar R

```
assert not is_x(r) ... severity failure;
VALIDATION_OK 256
rtl-results.txt → vectors.txt (entradas + esperado)
```

Los tiempos del testbench ordenan eventos; no constituyen STA ni cierre temporal.

GHDL genera Verilog para que Yosys pueda sintetizar el diseño

```
ghdl --synth --std=08 \
  -fsynopsys -fexplicit \
  --out=verilog INT8_16_DPU_perf_top

# stdout se guarda en design.v
```

CÓMO INTERPRETARLO

Entrada: VHDL analizado.

Salida: Verilog estructural.

Todavía no hay celdas de la biblioteca objetivo.

La preservación se comprueba después del mapeo.

La conversión es una transformación del diseño; requiere controles posteriores.

El script de Yosys hace explícita la política de síntesis y mapeo

```
read_liberty -lib cells.lib
read_verilog design.v
hierarchy -check -top INT8_16_DPU_perf_top
synth -top INT8_16_DPU_perf_top -flatten -noabc
dfflibmap -liberty cells.lib
abc -liberty cells.lib
delete t:$scopeinfo
clean
check -assert
write_json mapped.json
write_verilog -noattr mapped.v
tee -o stat.json stat -liberty cells.lib -json
```

CÓMO INTERPRETARLO

Una política común para comparar tecnologías.

Sin frecuencia objetivo.

Sin memoria SRAM mapeada.

Los scripts y versiones del caso forman parte de la evidencia.

check -assert revisa estructura; no prueba equivalencia funcional.

dfflibmap y ABC eligen celdas tecnológicas para funciones diferentes

Lógica genérica

Operadores, puertas y FF

→

dfflibmap

Estado → FF Liberty

→

ABC -liberty

Lógica → puertas Liberty

Netlist mapeada = instancias concretas + conexiones lógicas

Se mide el resultado de esa política de mapeo; no un mínimo matemático universal.

El cálculo de área puede reconstruirse celda por celda

Tipo de celda	Cantidad	Área unitaria · µm²	Subtotal · µm²
NOR2_X1	798	0,798	636,804
NOR4_X1	430	1,330	571,900
NAND4_X1	415	1,330	551,950
DFF_X1	96	4,522	434,112
Otros tipos combinacionales	4724	Variable	5.384,106
TOTAL	6463	—	7.578,872

Contar instancias × área de celda es distinto de estimar área con LUT, tamaño de archivo o nodo.

La clasificación distingue lógica, FF, latches y macros por semántica

```
counts = Counter(
  cell["type"] for cell in module["cells"].values()
)
unknown = set(counts) - set(cells)
if unknown: raise PipelineFailure(...)

subtotal = definition.area * count
categories[definition.category] += subtotal

# Decimal durante la suma; float al serializar JSON
```

CÓMO INTERPRETARLO

Se rechazan:

- Tipos sin mapear.
- Áreas inválidas o ausentes.
- Diseño vacío.
- Memorias pendientes.
- Estado fuera del perfil.

Una celda desconocida detiene el reporte; no se elimina para forzar un número.

La netlist se contrasta con el RTL usando modelos del mismo Liberty

```
yosys -s simulation_cells.ys
iverilog -g2012 -s area_validation_tb \
  -o mapped_sim validation_tb.v mapped.v cells_sim.v
vvp mapped_sim

if (r !== expected) $fatal(1, "Mapped mismatch...");
```

CÓMO INTERPRETARLO

Referencia: rtl-results.txt.

Comparación: bit a bit.

Se rechazan X/Z y diferencias.

No se cargan retardos SDF.

256 coincidencias respaldan los casos ejercitados; no equivalencia exhaustiva ni precisión ideal.

La confianza surge de controles complementarios, con límites distintos

Control	Qué permite sostener	Qué no prueba
SHA-256	Mismos bytes de entrada	Corrección original o certificación externa
Suma ↔ stat	Coherencia contable	Otra caracterización física independiente
RTL ↔ netlist	Preservación en 256 estímulos	Todos los estados o entradas posibles
Oráculo INT	Producto punto modular en esos casos	Exactitud matemática de FP/Posit/LNS
Pruebas de software	Contratos, errores y filtros	Área física real de un chip

El argumento sólido declara exactamente qué riesgo cubre cada evidencia.

El reporte conserva datos de cálculo y datos de procedencia

report.json / CSV

Áreas por categoría, instancias y biblioteca. Estado de validación explícito.

manifest.json

Fuentes, Liberty, versiones, política y hashes de estímulos y respuestas.

ZIP + SQLite

El ZIP conserva artefactos; SQLite mantiene trabajos y resultados persistidos.

Una gráfica debe poder volver al UUID, manifiesto, script y reporte por celda.

La comparación de la interfaz aplica filtros concretos

Entre tecnologías

La misma DPU; últimas corridas exitosas por biblioteca.

Entre formatos

Una biblioteca; última corrida exitosa de cada diseño.

Control que falta reforzar

Evitar mezclar versiones o políticas distintas solo porque coinciden los IDs.

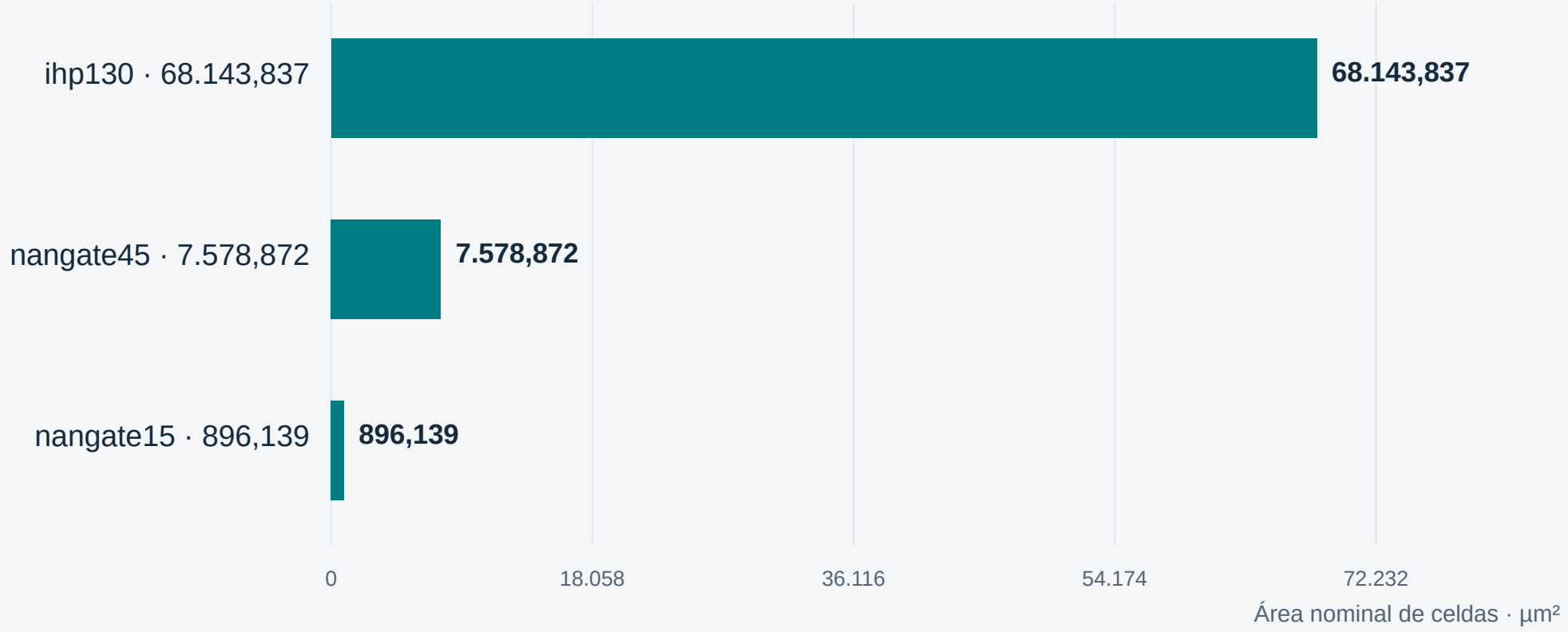
Una comparación visual correcta requiere también una cohorte de entradas y políticas compatible.

La evidencia completa alcanza 30 de las 33 combinaciones planeadas

DPU / formato	ihp130	nangate45	nangate15
int8_16	Documentado	Documentado	Documentado
int16_32	Documentado	Documentado	Documentado
fxp8_16	Documentado	Documentado	Documentado
fxp16_32	Documentado	Documentado	Documentado
fp8	Documentado	Documentado	Documentado
fp16	Documentado	Documentado	Documentado
fp32	Documentado	Documentado	Documentado
posit8	Documentado	Documentado	Documentado
posit16	Documentado	Documentado	Documentado
posit32	Documentado	Documentado	Documentado
lns16	Pendiente	Pendiente	Pendiente

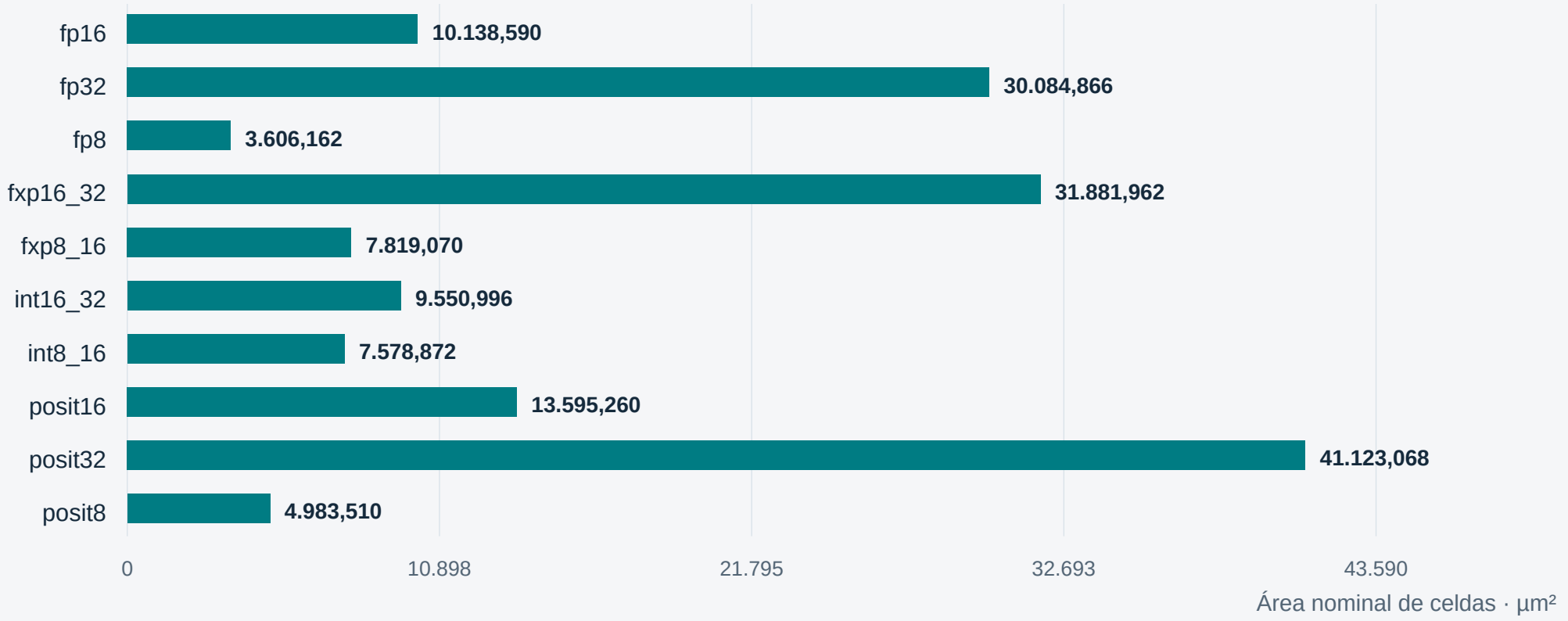
30 casos documentados no significa que LNS16 ni las TCU completas estén validadas.

El ejemplo INT8 muestra valores existentes y su fuente verificable



Es una comparación de bibliotecas e implementaciones; no de velocidad, potencia o silicio fabricado.

Dentro de Nangate45, el formato cambia la implementación y el área



LNS16 no se grafica: no existe un área final validada en el archivo.

La auditoría volvió a comprobar el archivo sin ejecutar EDA

44 entradas

Originales y snapshots coinciden con los SHA-256 registrados.

30 cuentas de área

Filas y categorías coinciden con Liberty y stat. Desacuerdo observado: 0.

1.536 resultados INT

Recalculados en Python. Además: 20 tests backend y 42 tests frontend aprobados.

Hoy se verificó el archivo conservado; no se volvieron a sintetizar los circuitos.

El archivo compacto permite auditar cuentas, pero no repetir toda la simulación

Conservado

Reportes, manifiestos, stat, scripts, vectores, testbenches y logs gzip.

No conservado

Netlists mapped.v / mapped.json, conversión design.v y modelos cells_sim.v.

Cierre recomendado

Retener el ZIP completo y sus hashes en futuras corridas controladas.

Los logs documentan una ejecución; no sustituyen los archivos necesarios para repetirla.

LNS16 expuso un límite de recursos del entorno de pruebas

Límite temporal

900 s por comando; código de salida y stdout/stderr al log.

Límite de memoria

Compose: 4 GB y 2 CPU. El runner local no impone límite de RAM.

LNS16 pendiente

No cerrar sus tres casos hasta evaluar recursos en ejecución acotada.

Un fallo por recursos no invalida los otros reportes; tampoco habilita a inventar el faltante.

La calidad del método depende de cerrar cuatro brechas concretas

Brecha	Por qué importa	Acción verificable
Artefactos / versiones	Repetir y revisar el resultado exacto	ZIP completo + hashes de código, imagen y script
LNS16 / recursos	Tres casos sin área publicada	Ejecutar con memoria y tiempo acotados
Oráculos no enteros	El RTL puede compartir un error con su referencia	Referencia numérica independiente si se afirma exactitud
Cohortes comparables	IDs iguales pueden ocultar cambios de biblioteca	Filtrar por hash y política de mapeo

Estas mejoras refuerzan las afirmaciones; no se presentan como implementadas hoy.

La conclusión debe nombrar implementación, biblioteca y nivel de prueba

“Estimación prelayout del área nominal de celdas de la DPU registrada, mapeada con la biblioteca y política documentadas.”

Lo sustentado

Sustentado: identidad de entradas, cuenta de celdas y comportamiento en los casos registrados.

Lo pendiente

Pendiente: cobertura LNS16, archivo completo y cualquier afirmación de precisión, timing o silicio.

Es defendible como estudio de área de celdas, con validación y límites expresamente acotados.

La revisión necesita decisiones sobre evidencia y alcance

Alcance

¿Se acepta área de celdas con wrapper, sin PnR ni objetivos de frecuencia?

Evidencia

¿Se exige ZIP completo, hashes de código/imagen y repetición controlada antes de publicar?

Ampliación

¿Se prioriza cerrar LNS16, estudiar otros corners o desarrollar oráculos numéricos?

El visto bueno debe referirse al método y a sus condiciones, no a una precisión física no demostrada.

Anexo A · El mapa de archivos conecta cada afirmación con su implementación

Responsabilidad	Archivo / símbolo
Perfil y tecnología	infrastructure/designs.py · catalog.py
Contrato de entrada	api/schemas.py · CreateExperimentRequest
Cola y estado	application/services.py · ExperimentService
Preparación VHDL	infrastructure/vhdl.py · analyze_sources
Oráculo y comparación	infrastructure/validation.py
Síntesis y reportes	infrastructure/synthesis.py · GhdlYosysSynthesis.run
Área y categorías	infrastructure/liberty.py
Ejecución externa	infrastructure/process_runner.py
Persistencia / descargas	infrastructure/sqlite_repository.py · api/routes.py
Filtros de gráficos	frontend/src/features/experiments/ComparisonPanel.tsx

Anexo B · Los comandos de simulación RTL quedan registrados

```
ghdl -a --std=08 -fsynopsys -fexplicit validation_tb.vhd
ghdl -e --std=08 -fsynopsys -fexplicit area_validation_tb
ghdl -r --std=08 -fsynopsys -fexplicit \
  area_validation_tb --assert-level=error \
  --ieee-asserts=disable-at-0

# Exigir VALIDATION_OK 256 y 256 salidas definidas
```

CÓMO INTERPRETARLO

-a: analizar.

-e: elaborar y enlazar.

-r: simular y fallar ante aserciones.

El log conserva comando, salida, exit code y duración.

Anexo C · La memoria y los registros tienen una política explícita

Registros

Incluidos; FF reales según la netlist. INT8: 96, INT16: 192 esperados.

ROM de corrección

Se implementa como lógica combinacional, no como macro SRAM.

Control de salida

Se rechazan memorias residuales, latches y macros inesperadas.

Anexo D · Las API permiten seguir y auditar un experimento

Endpoint	Uso / condición
GET /api/v1/catalog	Perfiles y bibliotecas habilitadas
POST /api/v1/experiments	Crea UUID; devuelve 202
GET /api/v1/experiments/{id}	Estado y reporte del caso
GET /api/v1/experiments/{id}/logs	Cola del log; hasta 100 KB
GET /api/v1/experiments/{id}/artifacts	ZIP; disponible al terminar o fallar
GET /api/v1/experiments/{id}/report.csv	Desglose por celda; exige succeeded
GET /api/health	Salud HTTP, no validez de área

Anexo E · Tabla trazable de áreas documentadas

DPU	IHP130 · µm²	Nangate45 · µm²	Nangate15 · µm²
int8_16	68,143.836600	7,578.872000	896.139264
int16_32	86,031.514800	9,550.996000	2,594.979840
fxp8_16	67,321.913400	7,819.070000	954.531840
fxp16_32	277,921.513800	31,881.962000	3,775.709184
fp8	31,708.870200	3,606.162000	983.040000
fp16	90,989.249400	10,138.590000	2,793.996288
fp32	269,112.186000	30,084.866000	8,200.962048
posit8	42,834.015000	4,983.510000	1,289.551872
posit16	118,561.816800	13,595.260000	3,649.781760
posit32	369,588.668400	41,123.068000	11,287.609344
lns16	PENDIENTE	PENDIENTE	PENDIENTE

Los decimales expresan la cuenta de biblioteca, no la incertidumbre de un chip real.

Anexo F · Release fijada no significa imagen completamente identificada

Suite fijada

OSS CAD Suite 2026-09-04 y SHA-256 del archivo descargado.

Versiones retenidas

GHDL 7.0.0-dev; Yosys 0.68+182; Icarus 14.0 devel.

Refuerzo recomendado

Digest de imagen, código y scripts por corrida; preservar los binarios o el contenedor.

Anexo G · La comprobación del build detecta fallos reales del toolchain

```
RUN apt-get install ... build-essential zlib1g-dev
USER area-lab
RUN python /app/scripts/check_toolchain.py

# Verifica: compilador + enlace nativo
# GHDL: análisis → elaboración → simulación → Verilog
```

CÓMO INTERPRETARLO

Un smoke test de instalación.

Detecta cc ausente y problemas de enlace.

Falla el build si el toolchain está roto.

No ejecuta la matriz de DPU.

Anexo H · Cómo repetir la auditoría sin sintetizar

```
python3 docs/presentaciones/revision_pipeline_2026-10-19/\
  auditar_evidencias.py

.venv/bin/pytest backend/tests -m "not integration" \
  -q -p no:cacheprovider

cd frontend
npm test
```

CÓMO INTERPRETARLO

Auditoría: 30 paquetes coherentes.

Pruebas ligeras: 20 backend.

Contratos e interfaz: 42 frontend.

Sin nuevas síntesis ni prueba de navegador.

Anexo I · El backend coordina y persiste; las herramientas EDA calculan

Tecnología	Función	Uso concreto en el programa
Python	Orquestación y cálculo	services.py, synthesis.py, liberty.py
Uvicorn / ASGI	Servidor HTTP del proceso	Docker CMD; un worker HTTP
FastAPI + Pydantic	Rutas y validación de solicitudes	IDs enumerados; OpenAPI
SQLite / WAL	Base embebida persistente	Estado y reportes por UUID
Docker / Compose	Entorno y límites declarados	Imagen, volumen y recursos

Anexo J · El frontend valida contratos y representa reportes reales

Tecnología	Función	Uso concreto en el programa
React	Componentes, propiedades y estado	Formulario, reporte y comparación
TypeScript	Tipos estáticos del código	No sustituye validar JSON en runtime
Zod	Validación de respuestas HTTP	schema.parse(data) antes de la UI
TanStack Query	Consultas, caché y polling	1,5 s con trabajos; 5 s sin trabajos
Vite	Desarrollo y build del frontend	No es otro servicio de producción

Anexo K · Referencias oficiales del procesamiento de hardware · 1

ID	Tema	URL oficial / primaria
F01	GHDL: análisis, elaboración y síntesis	https://ghdl.github.io/ghdl/using/Synthesis.html
F02	Yosys: síntesis genérica y mapeo tecnológico	https://yosyshq.readthedocs.io/projects/yosys/en/v0.54/cmd/synth.html
F03	Yosys: lectura de Liberty y modelos de simulación	https://yosyshq.readthedocs.io/projects/yosys/en/0.41/cmd/read_liberty.html
F04	Yosys: estadística de área	https://yosyshq.readthedocs.io/projects/yosys/en/v0.50/cmd/stat.html

Anexo K · Referencias oficiales del procesamiento de hardware · 2

ID	Tema	URL oficial / primaria
F05	Yosys: comprobaciones estructurales	https://yosyshq.readthedocs.io/projects/yosys/en/0.45/cmd/check.html
F06	Berkeley ABC: optimización y mapeo	https://people.eecs.berkeley.edu/~alanmi/abc/abc.htm
F07	Icarus Verilog y vvp: simulación de la netlist	https://steveicarus.github.io/iverilog/usage/getting_started.html
F18	EQY: equivalencia formal como verificación adicional	https://yosyshq.readthedocs.io/projects/eqy/en/latest/

Anexo L · Referencias de tecnología y plataforma · 1

ID	Tema	URL oficial / primaria
F08	IHP SG13G2: proceso real de 130 nm y estado del PDK abierto	https://github.com/IHP-GmbH/IHP-Open-PDK
F09	Nangate45 / FreePDK45: tecnología genérica predictiva	https://eda.ncsu.edu/freepdk/freepdk45/
F10	Nangate15 / FreePDK15: kit predictivo FinFET de investigación	https://eda.ncsu.edu/freepdk15/
F11	Unidades geométricas: documentación primaria y evidencia suministrada	https://ihp-open-pdk-docs.readthedocs.io/en/latest/layout_rules/01_general.html
F12	OSS CAD Suite: distribución de herramientas EDA	https://github.com/YosysHQ/oss-cad-suite-build

Anexo L · Referencias de tecnología y plataforma · 2

ID	Tema	URL oficial / primaria
F13	FastAPI: API y validación de solicitudes	https://fastapi.tiangolo.com/
F14	React: interfaz basada en componentes	https://react.dev/learn
F15	Vite: desarrollo y empaquetado del frontend	https://vite.dev/guide/
F16	SQLite: persistencia embebida	https://sqlite.org/serverless.html
F17	Docker y Compose: empaquetado y ejecución reproducible	https://docs.docker.com/get-started/docker-overview/

Anexo M · Fuentes de lenguajes, servidor y validadores · 1

ID	Tema	URL oficial / primaria
F19	Python: orquestación y procesos externos	https://docs.python.org/3.12/library/subprocess.html
F20	Pydantic: contratos de datos de la API	https://pydantic.dev/docs/validation/latest/concepts/models/
F21	Uvicorn: servidor ASGI	https://github.com/Kludex/uvicorn

Anexo M · Fuentes de lenguajes, servidor y validadores · 2

ID	Tema	URL oficial / primaria
F22	TypeScript: comprobación estática del frontend	https://www.typescriptlang.org/docs/handbook/intro.html
F23	Zod: validación de datos en ejecución	https://zod.dev/
F24	TanStack Query: datos remotos, caché y sondeo	https://tanstack.com/query/latest/docs/framework/react/overview